MPOO · GRUPO 6 · S23 · 2026-11-09

Interfaces y excepciones

Contratos que cumplen clases que no son familia, y programas que no truenan cuando algo sale mal (1.8 y U3)

100 minutos · Java 21 · FI-UNAM · 2027-1

Hoy cerramos polimorfismo con interfaces y abrimos el manejo de errores.

LA PREGUNTA DE HOY

¿Qué hacemos cuando las clases no son familia, y cuando algo sale mal?

Bloque	Qué resolvemos
Repaso	Polimorfismo en dos láminas
Interfaces	Contratos, y por qué una clase no tiene dos madres
Excepciones	Que el programa no truene y que una clase rechace datos malos
Proyecto final	Qué entregan y cuándo

Lo de interfaces quedó pendiente del 26 de octubre; el miércoles la P11 lo usa.

Repaso: polimorfismo

~5 min

Lo esencial del 26 de octubre, en dos láminas.

REPASO · POLIMORFISMO

Un tipo común y cada objeto responde a su manera

Notificacion.java

```java
class Notificacion {
    void enviar(String mensaje) {
        System.out.println("Notificación: " + mensaje);
    }
}
```

Email.java

```java
class Email extends Notificacion {
    @Override
    void enviar(String mensaje) {
        System.out.println("Email: " + mensaje);
    }
}
```

SMS.java

```java
class SMS extends Notificacion {
    @Override
    void enviar(String mensaje) {
        System.out.println("SMS: " + mensaje);
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Notificacion[] notificaciones = {
            new Email(), new SMS(), new Email()
        };

        for (Notificacion n : notificaciones) {
            n.enviar("Hola mundo");
        }
    }
}
```

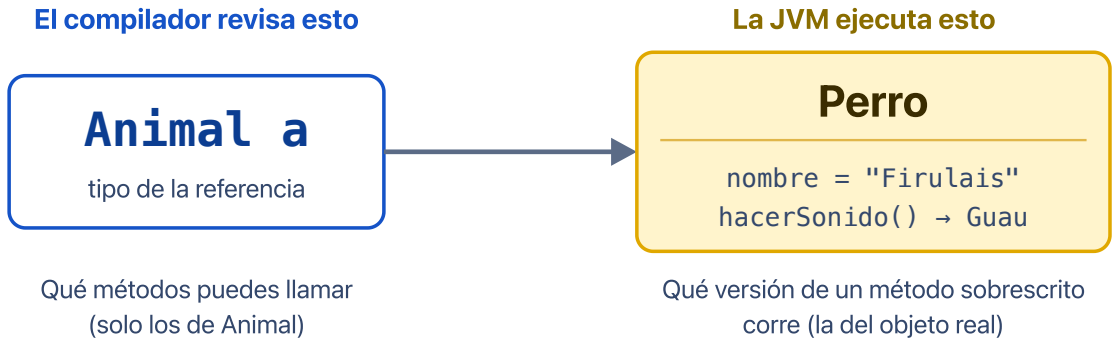
Consola

```
Email: Hola mundo
SMS: Hola mundo
Email: Hola mundo
```

Un arreglo de la madre y un for, sin ningún if que pregunte el tipo: así se ve el polimorfismo.

REPASO · LOS DOS TIPOS

La variable decide qué puedes llamar; el objeto, qué se ejecuta



- `Animal a = new Perro(...)` es válido: un Perro **es un** Animal.
- Al correr, la JVM ejecuta la versión del objeto real: **enlace dinámico**.
- Siempre `@Override`: si escribes mal el nombre, el compilador avisa.

Con esto basta para entender las interfaces.

Interfaces y herencia múltiple

⏱ ~28 min

¿Y si dos clases no comparten padre pero deben hacer lo mismo?

INTERFACES · EL PROBLEMA

Una Sirena también avisa, pero no es una Notificacion

- `Email` y `SMS` heredan de `Notificacion`.
- Una `Sirena` también «envía un aviso», pero no tiene sentido que sea una Notificacion ni que herede de Email.
- Y una clase solo puede tener **una** madre.
- Queremos el polimorfismo sin obligar a nadie a heredar.

Necesitamos algo que diga qué debe saber hacer una clase, sin decir cómo.

INTERFACES · LA IDEA

Una interfaz es un contrato: dice qué, no cómo

Notificador.java

```
interface Notificador {
    void enviar(String mensaje);
}
```

- Solo declara métodos: **sin cuerpo**.
- La clase firma el contrato con `implements` y escribe cada método.
- Los métodos de una interfaz son `public`; al implementarlos también.

> **ANALOGÍA**
> Un contrato de renta: dice qué debe cumplir quien renta, no cómo organiza su casa.

El contrato no tiene código: cada clase pone el suyo.

INTERFACES · EN CÓDIGO

Email y Sirena cumplen el contrato, sin ser familia

Notificador.java

```java
interface Notificador {
    void enviar(String mensaje);
}
```

Email.java

```java
class Email implements Notificador {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Email: " + mensaje);
    }
}
```

Sirena.java

```java
class Sirena implements Notificador {
    @Override
    public void enviar(String mensaje) {
        System.out.println("Sirena: ¡" + mensaje + "!");
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Notificador[] canales = {
            new Email(), new Sirena()
        };

        for (Notificador c : canales) {
            c.enviar("Incendio");
        }
    }
}
```

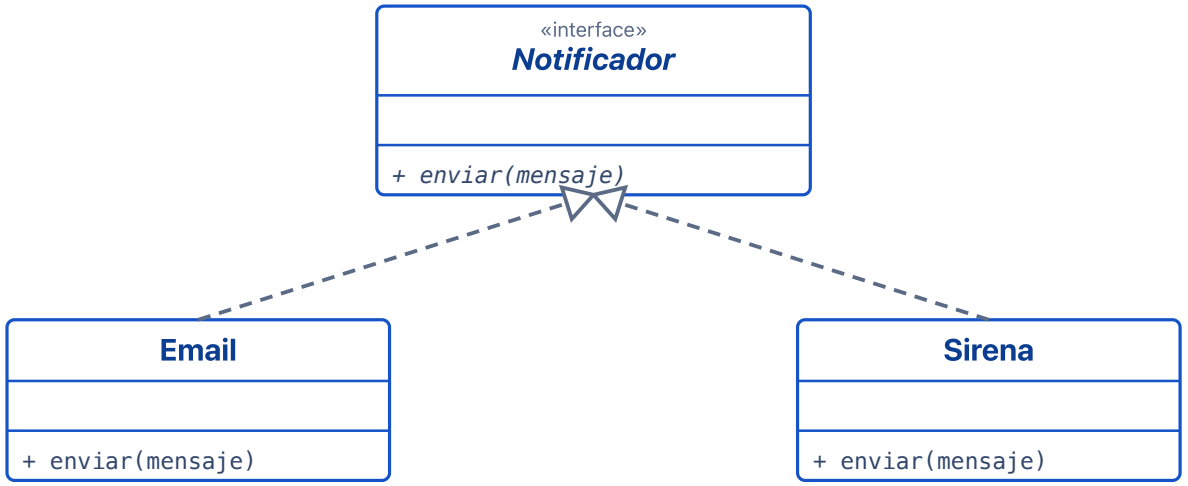
Consola

```
Email: Incendio
Sirena: ¡Incendio!
```

Mismo for, mismo tipo, dos clases que no son familia.

INTERFACES · EN UN DIAGRAMA

La interfaz se marca con «interface» y flecha punteada



Línea	Significa
Continua, triángulo hueco	`extends`
Punteada, triángulo hueco	`implements`

Continua: es un. Punteada: cumple el contrato de.

INTERFACES · REGLA 1

Una interfaz no se instancia: es un tipo, no un objeto

Notificador.java

```
interface Notificador {
    void enviar(String mensaje);
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Notificador n = new Notificador();

        n.enviar("Hola");
    }
}
```

javac

```
Main.java:3: error: Notificador is
  abstract; cannot be instantiated
  Notificador n = new Notificador();
                  ^
```

Sí puede ser el tipo de una variable; lo que no se puede es crear un objeto con new.

INTERFACES · REGLA 2

Quien firma el contrato debe cumplirlo completo

Notificador.java

```
interface Notificador {
    void enviar(String mensaje);
}
```

Email.java

```
class Email implements Notificador {
    void avisar(String mensaje) {
        System.out.println("Email: " + mensaje);
    }
}
```

javac

```
Email.java:1: error: Email is not
  abstract and does not override
  abstract method enviar(String)
  in Notificador
```

Falta enviar(String): sin él, Email no cumple el contrato y no compila.

INTERFACES · LA TABLA

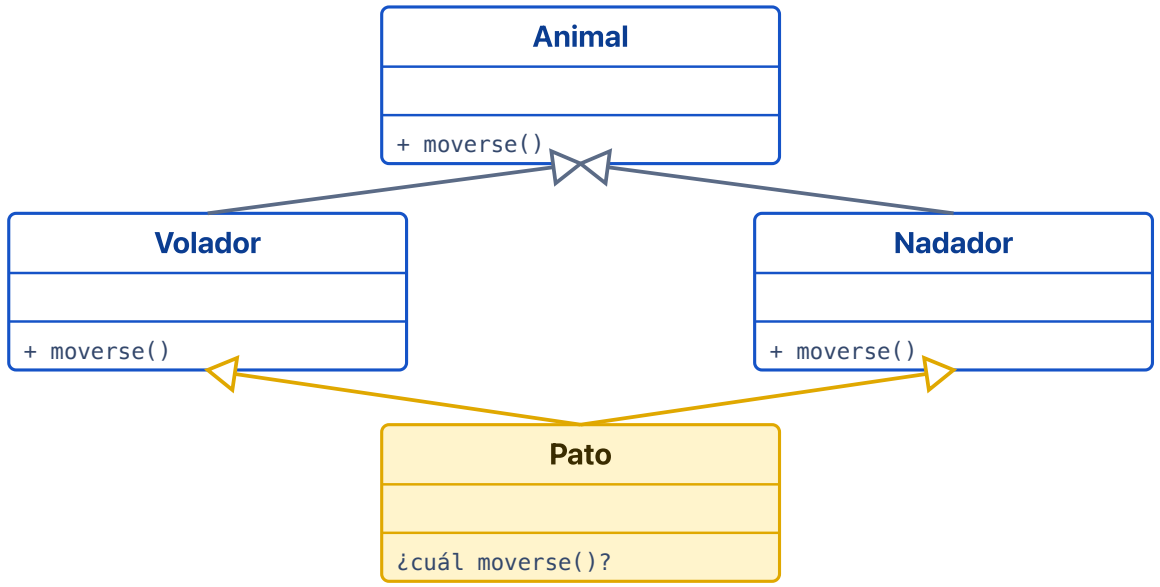
Interfaz y clase abstracta: cuándo cada una

	Clase abstracta	Interfaz
Palabras	`abstract class` y `extends`	`interface` e `implements`
Cuántas puede tener una clase	**Una sola madre**	**Todas las que quiera**
Atributos y constructor	Sí	No (solo constantes)
Métodos	Con cuerpo y sin cuerpo	Sin cuerpo (y `default`, que sigue)
Úsala cuando	Hay un «es un» con código compartido	Una capacidad que clases distintas deben ofrecer

Abstracta: lo que son. Interfaz: lo que saben hacer.

HERENCIA MÚLTIPLE · EL PROBLEMA

Un Pato vuela y nada: ¿le damos dos padres?



- `Volador` y `Nadador` heredan de `Animal`.
- Los dos sobrescriben `moverse()`.
- Un `Pato` querría ser las dos cosas.

Esta forma de rombo es el problema del diamante.

HERENCIA MÚLTIPLE · JAVA DICE QUE NO

En Java una clase tiene un solo extends

Animal.java

```java
class Animal {
    void moverse() {
        System.out.println("Se mueve");
    }
}
```

Volador.java

```java
class Volador extends Animal {
    @Override
    void moverse() {
        System.out.println("Vuela");
    }
}
```

Nadador.java

```java
class Nadador extends Animal {
    @Override
    void moverse() {
        System.out.println("Nada");
    }
}
```

Pato.java

```java
class Pato extends Volador, Nadador {
}
```

javac

```
Pato.java:1: error: '{' expected
class Pato extends Volador, Nadador {
                                    ^
```

Java prohíbe la herencia múltiple de clases.

HERENCIA MÚLTIPLE · POR QUÉ

¿Qué moverse() heredaría Pato?

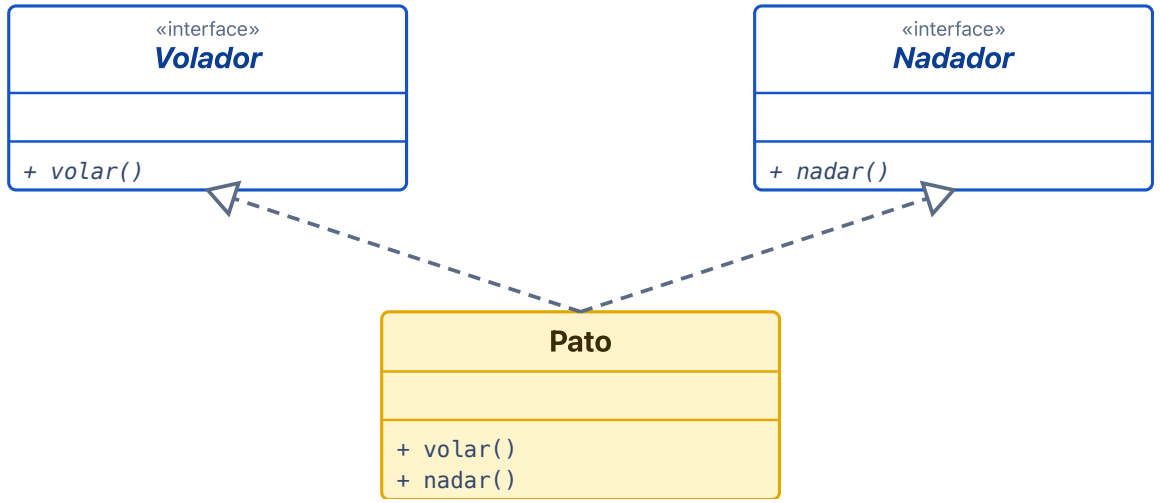
Clase	moverse() imprime
Volador	Vuela
Nadador	Nada
Pato (hereda de las dos)	**¿Vuela o Nada?**

Con dos versiones heredadas del mismo nombre, el compilador no puede elegir por ti. Java lo evita prohibiendo el segundo `extends`.

Ese choque entre dos versiones es el problema del diamante.

HERENCIA MÚLTIPLE · LA SALIDA

Una clase, muchas interfaces



- Una clase solo puede tener un `extends`.
- Pero puede tener todos los `implements` que necesite.
- Una interfaz no trae código: no hay versiones que choquen.

De clases no; de interfaces sí.

HERENCIA MÚLTIPLE · EN CÓDIGO

Pato implementa las dos y se usa como cada una

Volador.java

```java
interface Volador {
    void volar();
}
```

Nadador.java

```java
interface Nadador {
    void nadar();
}
```

Pato.java

```java
class Pato implements Volador, Nadador {
    @Override
    public void volar() {
        System.out.println("El pato vuela");
    }

    @Override
    public void nadar() {
        System.out.println("El pato nada");
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Pato pato = new Pato();

        Volador v = pato;
        Nadador n = pato;

        v.volar();
        n.nadar();
    }
}
```

Consola

```
El pato vuela
El pato nada
```

El mismo objeto sirve como Volador y como Nadador.

HERENCIA MÚLTIPLE · MISMO MÉTODO

Si las dos interfaces piden el mismo método, se escribe una vez

Volador.java

```java
interface Volador {
    void moverse();
}
```

Nadador.java

```java
interface Nadador {
    void moverse();
}
```

Pato.java

```java
class Pato implements Volador, Nadador {
    @Override
    public void moverse() {
        System.out.println("El pato vuela y nada");
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Pato pato = new Pato();

        Volador v = pato;
        Nadador n = pato;

        v.moverse();
        n.moverse();
    }
}
```

Consola

```
El pato vuela y nada
El pato vuela y nada
```

No hay choque: las dos piden moverse() y Pato lo escribe una sola vez.

HERENCIA MÚLTIPLE · DEFAULT

Con métodos default, el choque vuelve

Volador.java

```
interface Volador {
    default void moverse() {
        System.out.println("Vuela");
    }
}
```

Nadador.java

```
interface Nadador {
    default void moverse() {
        System.out.println("Nada");
    }
}
```

Pato.java

```
class Pato implements Volador, Nadador {
}
```

javac

```
Pato.java:1: error: types Volador and
  Nadador are incompatible;
  class Pato inherits unrelated
  defaults for moverse() from types
  Volador and Nadador
```

Una interfaz puede traer un default con cuerpo; si dos traen el mismo, Java obliga a elegir.

HERENCIA MÚLTIPLE · DEFAULT, LA SALIDA

Pato decide con Volador.super y Nadador.super

Volador.java

```java
interface Volador {
    default void moverse() {
        System.out.println("Vuela");
    }
}
```

Nadador.java

```java
interface Nadador {
    default void moverse() {
        System.out.println("Nada");
    }
}
```

Pato.java

```java
class Pato implements Volador, Nadador {
    @Override
    public void moverse() {
        Volador.super.moverse();
        Nadador.super.moverse();
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        new Pato().moverse();
    }
}
```

Consola

```
Vuela
Nada
```

La clase sobrescribe moverse() y elige a quién llamar.

HERENCIA MÚLTIPLE · RESUMEN

Qué se puede y qué no en Java

Quieres...	¿Se puede?	Cómo
Heredar de dos clases	No	Solo un `extends`
Cumplir dos contratos	**Sí**	**`implements A, B`**
Reusar código de varios lados	Con cuidado	`default`, o composición (S18)

Si dudas entre heredar y reusar, pregunta: ¿es un, o tiene un?

Excepciones: lo básico

~50 min

¿Qué pasa cuando algo sale mal mientras el programa corre?

EXCEPCIONES · CUANDO TRUENA

Leer la traza: qué pasó, con qué dato y en qué línea

Main.java

```java
public class Main {
    public static void main(String[] args) {
        String texto = "dos";
        System.out.println("Convirtiendo...");
        int piezas = Integer.parseInt(texto);
        System.out.println("Piezas: " + piezas);
    }
}
```

Consola

```
Convirtiendo...
Exception in thread "main" java.lang.NumberFormatException:
    For input string: "dos"
  at java.base/java.lang.Integer.parseInt(Integer.java:662)
  at Main.main(Main.java:5)
```

Renglón	Qué te dice
`NumberFormatException`	**Qué** pasó: un texto que no es número
`For input string: "dos"`	**Con qué dato**: el mensaje
`at Main.main(Main.java:5)`	**Dónde**: tu archivo y tu línea
`at java.base/...`	Código de Java: sáltalo y busca tu archivo

La línea 6 nunca corrió: el programa terminó en la 5.

EXCEPCIONES · QUÉ ES

Una excepción es un objeto que avisa que algo salió mal

- Cuando algo falla al correr, Java **crea un objeto** con el tipo de error y un mensaje.
- Ese objeto se **lanza**: sube buscando a alguien que lo atrape.
- Si nadie lo atrapa, llega hasta `main` y **el programa termina** con la traza.

ANALOGÍA

Una alarma de incendio: alguien la activa en un piso y suena hasta que alguien la atiende. Si nadie la atiende, se evacua todo el edificio.

Excepción no es un error de compilación: el programa compiló y truena al correr.

EXCEPCIONES · YA LAS CONOCEN

Las que ya les han tronado tienen nombre

Excepción	Cuándo sale	Mensaje de Java
`ArrayIndexOutOfBoundsException`	`precios[3]` en un arreglo de 3 (P4)	`Index 3 out of bounds for length 3`
`NullPointerException`	Usar una referencia que vale `null` (S09)	`Cannot invoke "String.length()"...`
`NumberFormatException`	`Integer.parseInt("dos")`	`For input string: "dos"`
`InputMismatchException`	`sc.nextInt()` cuando escriben letras	`null` (sin mensaje)
`ArithmeticException`	`10 / 0` con enteros	`/ by zero`

Todas pasan al correr, con datos que el compilador no podía conocer.

EXCEPCIONES · TRY-CATCH

try-catch: intenta, y si falla, atrapa

Main.java

```java
public class Main {
    public static void main(String[] args) {
        String texto = "dos";
        try {
            System.out.println("Convirtiendo...");
            int piezas = Integer.parseInt(texto);
            System.out.println("Piezas: " + piezas);
        } catch (NumberFormatException e) {
            System.out.println("Eso no es un número: " + texto);
        }
        System.out.println("El programa sigue");
    }
}
```

- En `try` va el código que puede fallar.
- Si una línea lanza la excepción, **las que siguen dentro del try se saltan** (la gris).
- Corre el `catch` del tipo que coincide.
- Después, el programa **sigue** normal.

Consola

```
Convirtiendo...
Eso no es un número: dos
El programa sigue
```

Mismo error que hace un momento, pero ahora el programa no termina.

Pedir la opción hasta que sea válida

Main.java

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion = 0;
        boolean valida = false;
        while (!valida) {
            System.out.print("Opción (1 a 3): ");
            try {
                opcion = Integer.parseInt(sc.nextLine());
                if (opcion >= 1 && opcion <= 3) {
                    valida = true;
                } else {
                    System.out.println("Solo de 1 a 3");
                }
            } catch (NumberFormatException e) {
                System.out.println("Escribe un número");
            }
        }
        System.out.println("Elegiste la " + opcion);
    }
}
```

Consola

```
Opción (1 a 3): dos
Escribe un número
Opción (1 a 3): 7
Solo de 1 a 3
Opción (1 a 3): 2
Elegiste la 2
```

- Se lee la línea completa con `nextLine()` y se convierte con `parseInt`.
- El `catch` atrapa el texto que no es número; el `if`, el número fuera de rango.
- El `while` vuelve a preguntar.

Este es el menú que pide el proyecto: escriban lo que escriban, no truena.

EXCEPCIONES · ERROR COMÚN

El catch vacío esconde el problema

Main.java

```java
public class Main {
    public static void main(String[] args) {
        String texto = "dos";
        int opcion = 0;
        try {
            opcion = Integer.parseInt(texto);
        } catch (NumberFormatException e) {
        }
        System.out.println("Elegiste la " + opcion);
    }
}
```

Consola

```
Elegiste la 0
```

- Compila y no truena, pero **nadie se entera** de que el dato era malo.
- `opcion` se queda en 0 y el programa sigue como si nada.
- En el `catch` siempre haz algo: avisa al usuario o vuelve a preguntar.

Un programa que truena se arregla; uno que falla en silencio, no se nota.

EXCEPCIONES · ANTES

Un constructor que solo imprime «error» no protege nada

Producto.java

```java
public class Producto {
    private String nombre;
    private int precio;

    public Producto(String nombre, int precio) {
        if (precio <= 0) {
            System.out.println("Error: precio inválido");
        }
        this.nombre = nombre;
        this.precio = precio;
    }

    public int getPrecio() {
        return precio;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto("papas", -18);
        System.out.println("Cobro: $" + p.getPrecio());
    }
}
```

Consola

```
Error: precio inválido
Cobro: $-18
```

Avisó, pero el producto se creó con precio -18 y la máquina cobra eso.

EXCEPCIONES · THROW

throw: la clase se niega a crear un objeto inválido

Producto.java

```java
public class Producto {
    private String nombre;
    private int precio;

    public Producto(String nombre, int precio) {
        if (precio <= 0) {
            throw new IllegalArgumentException("Precio inválido: " + precio);
        }
        this.nombre = nombre;
        this.precio = precio;
    }

    public int getPrecio() {
        return precio;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto("papas", -18);
        System.out.println("Cobro: $" + p.getPrecio());
    }
}
```

Consola

```
Exception in thread "main"
java.lang.IllegalArgumentException:
    Precio inválido: -18
  at Producto.<init>(Producto.java:7)
  at Main.main(Main.java:3)
```

Con throw el producto con precio -18 nunca existe. IllegalArgumentException significa «me pasaste un argumento que no acepto».

EXCEPCIONES · QUIÉN ATRAPA

La clase lanza; quien la usa atrapa

Main.java

```java
public class Main {
    public static void main(String[] args) {
        try {
            Producto p = new Producto("papas", -18);
            System.out.println("Cobro: $" + p.getPrecio());
        } catch (IllegalArgumentException e) {
            System.out.println("No se pudo: " + e.getMessage());
        }
        System.out.println("La máquina sigue");
    }
}
```

Consola

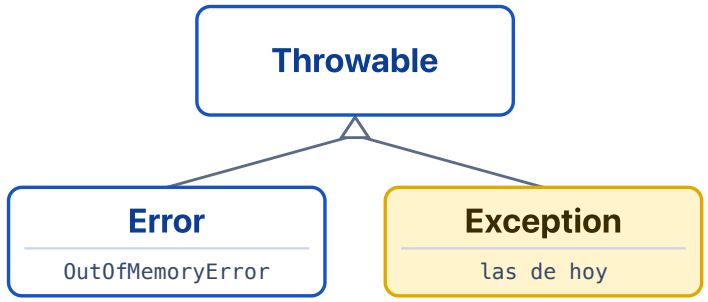
```
No se pudo: Precio inválido: -18
La máquina sigue
```

Quién	Qué hace
`Producto`	Revisa el dato y lanza con `throw` si no sirve
`Main`	Atrapa con `try-catch` y decide qué hacer
`e.getMessage()`	Regresa el texto que se puso al lanzar

Producto no sabe si hay un usuario o un menú: solo dice que no. Main decide cómo avisar.

VÁLVULA · EL MAPA

Todas son hijas de Throwable



Rama	Qué es	¿Se atrapa?
`Error`	La JVM tiene un problema grave (se quedó sin memoria)	No
`Exception`	**Tu programa recibió algo que no esperaba**	**Sí**

Las cinco de la tabla y IllegalArgumentException están del lado de Exception.

VÁLVULA · FINALLY

finally corre siempre, falle o no

Main.java

```java
public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Abro la caja");
            int piezas = Integer.parseInt("dos");
            System.out.println("Piezas: " + piezas);
        } catch (NumberFormatException e) {
            System.out.println("Dato inválido");
        } finally {
            System.out.println("Cierro la caja");
        }
    }
}
```

Consola

```
Abro la caja
Dato inválido
Cierro la caja
```

- Si el `try` sale bien: try, finally.
- Si falla: try, catch, finally.
- Sirve para cerrar lo que abriste: un archivo, un `Scanner`.

Lo que va en finally pasa en los dos caminos.

EXCEPCIONES · RESUMEN

Dos herramientas y un error que evitar

Situación	Qué usas	Ejemplo de hoy
El usuario escribe algo que no sirve	`try-catch` y volver a preguntar	El menú con `parseInt`
Tu clase recibe un dato que no acepta	`throw new IllegalArgumentException(...)`	Producto con precio negativo
Atrapar y no hacer nada	Nunca: el error se esconde	El catch vacío

try-catch para lo que llega de afuera; throw para lo que tu clase no acepta.

Proyecto final

~10 min

Lo que entregan el 25 de noviembre.

PROYECTO · QUÉ CONSTRUYEN

Una app de consola que modele un problema real con objetos

#	Requisito	Lo vimos en
1	**Encapsulamiento: 4 clases con `private` y métodos que validan**	**S18 y P9**
2	**Herencia: una clase abstracta con 2 hijas**	**S16, P7 y P8**
3	**Polimorfismo: un arreglo o lista de la madre, sin `if` por tipo**	**S20 y P10**
4	**Interfaz propia, implementada por 2 clases**	**Hoy y P11**
5	**Excepciones: el menú no truena y la validación lanza**	**Hoy**
6	Menú con `Scanner` que valida	Hoy
7	Paquetes con `package` e `import`	S18

Los cinco amarillos son el mínimo para aprobar. Equipos de 3 o 4.

PROYECTO · QUÉ ENTREGAN

Solo dos cosas: la liga al repo y el reporte en PDF

En la Discussion #115

- En el comentario de su equipo.
- **La liga al repo** de GitHub, público.
- **El reporte en PDF**, adjunto ahí mismo.
- Opcional: sus diapositivas de apoyo para la presentación.

El reporte

- Portada, el problema y el diagrama de clases.
- Una tabla de dónde está cada concepto.
- Cómo correrlo, qué falta y quién hizo qué.

Código 15, reporte 5, presentación 5 y trabajo en equipo 5: 30 puntos del curso.

PROYECTO · FECHAS

Dos semanas, con el parcial en medio

Fecha	Qué pasa
Miércoles 18 de noviembre	Parcial 2 (U2 y U3), en el salón
Lunes 23 de noviembre	Asesoría en clase: traigan lo que lleven
Miércoles 25 de noviembre, antes de las 15:00	**Liga y reporte en la Discussion #115; presentaciones de 8 a 10 minutos**

En la presentación le pregunto a cualquiera del equipo: cada quien debe poder explicar todo el código.

LO QUE SIGUE

Miércoles 11 · P11 Polimorfismo, segunda parte, con interfaces

Idea de hoy	En una línea
Interfaz	Contrato sin código: `implements`, todas las que quieras
Herencia múltiple	De clases no (un solo `extends`); de interfaces sí
Excepciones	`try-catch` para lo que llega de afuera; `throw` para lo que tu clase no acepta

El miércoles lo practican en la P11.